1. A/B Testing & Experimentation Framework

Project Objective

Build a system that compares two versions of a product feature and determines which version performs better using statistical analysis.

Example

An e-commerce website has:

• Version A: Old checkout button
• Version B: New checkout button
We want to determine whether Version B actually improves the conversion rate .

Technology Requirements

• Python
• Pandas
• NumPy
• Matplotlib / Seaborn
• SciPy
• SQL
• Jupyter Notebook / Google Colab
• Power BI — optional

Project Flow

Dataset
↓
Data Cleaning
↓
Exploratory Data Analysis
↓
Separate Group A & Group B
↓
Calculate KPIs
↓

Conversion Rate Comparison
↓
Hypothesis Testing
↓
Statistical Significance
↓
Business Recommendation
↓
Dashboard / Final Report
Step 1 — Dataset

Students can work with columns like:

User_ID

Group

Device

Visits

Purchases

Revenue

U001

A

Mobile

5

1

1200

U002

B

Desktop

4

2

2500

U003

A

Mobile

6

0

0

Dataset can contain 5,000–20,000 users .

Step 2 — Data Cleaning

Students should:

• Check missing values
• Remove duplicates
• Check incorrect values
• Handle outliers
• Validate Group A and Group B
• Check data types

Step 3 — KPI Analysis

Calculate:

• Total Users
• Total Purchases
• Conversion Rate
• Average Revenue
• Revenue per User
• Click-Through Rate
Example:

Conversion Rate =
Number of Purchasers / Total Users × 100
Step 4 — A/B Comparison

Compare:

Group A Conversion Rate = 8.2 %

Group B Conversion Rate = 9.7 % But students cannot simply conclude B is better .

They must perform statistical testing.

Step 5 — Hypothesis Testing

Students define:

H0 = There is no significant difference between A and B .

H1 = There is a significant difference between A and B . Use:

• Z-test / Chi-square test
• p-value
• Significance level = 0.05

Step 6 — Final Decision

Example:

p - value = 0.021
alpha = 0.05

0.021 < 0.05 Therefore:

Reject H0 Conclusion:

Version B has a statistically significant improvement in conversion rate.

Final Deliverables

Students should submit:

- Dataset
- Python notebook
- SQL queries
- EDA report
- A/B testing analysis
- Statistical test results
- Dashboard
- Final business recommendation
- Project documentation
- Presentation

2. Anomaly Detection System for IT Infrastructure

Project Objective

Build a monitoring system that identifies unusual server/network behavior and alerts the IT team.

Example

Normally:

CPU Usage = 40 – 70 %
Memory = 50 – 75 %
Network Traffic = Normal
Response Time = 100 – 300 ms Suddenly:

CPU = 98 %

Memory = 96 %
Response Time = 5000 ms The system should identify this as an anomaly .

Technology Requirements

- Python
- Pandas
- NumPy
- Scikit-learn
- Matplotlib
- SQL
- Machine Learning
- Power BI — optional

Project Flow

Server Logs / Monitoring Data
↓
Data Collection
↓
Data Cleaning
↓
Feature Engineering
↓
EDA
↓
Normal vs Abnormal Pattern
↓
Anomaly Detection Model
↓
Anomaly Score
↓
Alert Generation
↓
Dashboard
↓
Final Report
Step 1 — Dataset

Create/use a dataset containing:

Timestamp

Server

CPU

Memory

Network

Response_Time

10:00

Server01

45

62

350

120

10:05

Server01

51

65

400

140

10:10

Server01

97

95

2500

4200

Students can generate synthetic monitoring data using Python.

Step 2 — Data Cleaning

Check:

• Missing values
• Duplicate logs
• Invalid timestamps
• Negative values
• Extreme values

Step 3 — Feature Engineering

Create features such as:

• CPU utilization
• Memory utilization
• Network traffic
• Response time
• Requests per minute
• Error rate

Step 4 — Visualization

Students create:

• CPU usage trend
• Memory usage trend
• Network traffic trend
• Response-time trend
This helps understand normal behavior.

Step 5 — Anomaly Detection

Students can implement:

Beginner:

• Z-score
• IQR
Intermediate:

• Isolation Forest
Advanced:

• Local Outlier Factor
• One-Class SVM
For internship, Isolation Forest is a good main ML approach.

Step 6 — Anomaly Detection

Example output:

Timestamp

Server

CPU

Memory

Anomaly

10:00

S01

45

62

Normal

10:05

S01

51

65

Normal

10:10

S01

97

95

Anomaly

Step 7 — Alert System

Students can create a simple alert:

■ ALERT

Server : Server01
CPU Usage : 97 %
Memory Usage : 95 %
Response Time : 4200 ms

Status : ANOMALY DETECTED For an advanced version, email notification can be added.

Step 8 — Dashboard

Power BI dashboard can contain:

• Total Servers
• Healthy Servers
• Anomalies Detected
• Average CPU
• Average Memory
• Response Time
• Anomaly Trend
• Server-wise anomalies

Final Deliverables

• Dataset
• Python notebook
• SQL queries
• ML model
• Anomaly detection results
• Alert module
• Power BI dashboard
• Documentation
• Presentation
• GitHub/project folder

3. Search Relevance & Semantic Search Optimization

This is the most advanced project among these three.

Project Objective

Build a search system that understands the meaning of the user's query , rather than matching only exact keywords.

Example

User searches:

"cheap running shoes" Traditional keyword search may return:

Cheap Running Shoes Semantic search should also understand:

Budget Sports Shoes
Affordable Jogging Shoes
Low Price Running Sneakers
Technology Requirements

• Python
• Pandas
• NumPy
• SQL
• NLP
• Scikit-learn
• TF-IDF
• Cosine Similarity
• Sentence Embeddings
• Vector Search
• Elasticsearch/OpenSearch — optional
• Streamlit — optional

Project Flow

Product Dataset
↓
Data Cleaning
↓
Text Preprocessing
↓
Create Product Text
↓
Traditional Keyword Search
↓
TF - IDF Search
↓

Semantic Embeddings
↓
Similarity Calculation
↓
Ranking
↓
Search Results
↓
Evaluate Search Quality
↓
Optimize Search
↓
Final Search Application
Step 1 — Dataset

Example:

Product_ID

Product_Name

Category

Description

Price

P001

Nike Running Shoe

Shoes

Lightweight running shoe

4500

P002

Adidas Sports Shoe

Shoes

Comfortable sports footwear

3800

P003

Puma Casual Shoe

Shoes

Casual everyday footwear

2500

Dataset should ideally contain 5,000+ products .

Step 2 — Text Preparation

Combine:

Product Name
+
Category
+
Description
+
Brand Example:

Nike Running Shoe Shoes Lightweight running shoe Nike
Step 3 — Traditional Search

First build basic keyword search.

Example:

Query :
"running shoes" Search based on matching keywords.

Step 4 — TF-IDF

Convert text into numerical vectors using:

TF - IDF Then calculate:

Cosine Similarity to rank products.

Step 5 — Semantic Search

Now introduce embeddings.

Example:

Query :
"affordable shoes for jogging" The system should understand that:

jogging ≈ running
affordable ≈ cheap / budget and return relevant products even when exact words don't match.

Step 6 — Ranking

Create ranking based on:

Semantic Similarity
+
Keyword Relevance
+
Product Rating
+
Popularity Example:

Product

Similarity

Rating

Final Rank

Nike Running Shoe

0.92

4.6

1

Adidas Sports Shoe

0.87

4.5

2

Puma Casual Shoe

0.54

4.2

3

Step 7 — Search Evaluation

Students should evaluate the search engine using metrics such as:

- Precision
- Recall
- F1 Score
- CTR

• MRR
• NDCG

Step 8 — Build Search UI

For internship, students can create a simple:

Streamlit Search Application

-- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
PRODUCT SEARCH
-- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --

Search : [affordable running shoes]

[SEARCH]

Results :

1. Nike Running Shoe
■ 4 , 500
■ 4.6

2. Adidas Sports Shoe
■ 3 , 800
■ 4.5
-- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
Final Deliverables

• Product dataset
• SQL database
• Data-cleaning notebook
• NLP preprocessing
• TF-IDF search
• Semantic search
• Ranking algorithm
• Search evaluation
• Streamlit application

- Documentation
- Presentation
- GitHub repository